Gestión de Incidencias de Red asistida por IA

Reto técnico · AI Engineer · HACOM

Rubén Castillo

Octubre 2026

Table of Contents

1. Resumen ejecutivo 2
Términos 2
2. Supuestos 2
3. Matriz de decisiones de automatización 3
Señales de escalamiento al Operador NOC 4
4. Arquitectura conceptual / lógica 4
4.1 Componentes, datos y límites de acceso 5
4.2 Memoria 5
4.3 Estados, fallos y escalamiento del workflow 6
4.4 El agente de investigación 6
5. Arquitectura física (AWS, híbrida) 7
6. Seguridad: matriz de riesgos y controles 9
7. Estrategia de evaluación de la IA generativa 10
7.1 Qué se evalúa 10
7.2 Tres capas 10
7.3 Despliegue gradual 11
8. Operación y limitaciones 11
8.1 Latencia y costo (estimación) 11
8.2 Observabilidad 11
8.3 Resiliencia 12
8.4 Supervisión humana 12
8.5 Riesgos residuales 12
8.6 Pendientes antes de producción 12
9. Decisiones técnicas principales 13
Anexo A. Prototipo 13

1. Resumen ejecutivo

La propuesta es un **workflow determinista** que orquesta las 8 actividades del proceso. Solo hay un **agente IA**, limitado a la investigación de causas probables, y dos **llamadas LLM puntuales** dentro del workflow: una para extraer campos de texto libre y otra para redactar el resumen técnico. Todo lo que tiene reglas conocidas queda fuera del LLM: validación, correlación, prioridad, SLA, ruteo, escritura en el ticket y escalamiento.

La garantía de que *la IA no ejecuta cambios sobre la red* no depende del prompt, sino de la infraestructura:

- El agente solo accede a tools de solo lectura, expuestas por un servidor MCP con un rol IAM de lectura.
- La única ruta de escritura (el Ticket maestro) pertenece al workflow, con un rol distinto.
- Toda salida generativa pasa una validación determinista (esquema, citas verificables y políticas) antes de llegar al Operador NOC.

La solución corre en **AWS**, en topología híbrida: los sistemas del operador permanecen on-premise y se consultan por Direct Connect. Si la IA falla, la operación se degrada sin detenerse: los pasos deterministas siguen funcionando y la Incidencia se deriva por reglas, marcada "sin análisis IA".

Qué está implementado y qué no. Este documento es un diseño. El prototipo adjunto contiene los contratos de tools, el esquema de salida, las reglas deterministas y un set de casos de evaluación ejecutable. No se desplegó ningún servicio ni se invocó ningún modelo real.

Términos

Término	Significado
Alerta	Evento automático del sistema de monitoreo.
Ticket	Registro en la plataforma de tickets, abierto por un usuario o por un sistema.
Incidencia	Unidad de trabajo que se clasifica, investiga y deriva. Agrupa 1..N Alertas y Tickets correlacionados.
Ticket maestro	Ticket que actúa como registro oficial de la Incidencia.
Operador NOC	Persona de operaciones que revisa, aprueba y escala.
Equipo resolutor	Equipo técnico al que se deriva la Incidencia (Core IP, Transporte, Acceso FTTH, etc.).

2. Supuestos

#	Supuesto	Impacto si no se cumple
S1	Los datos de incidencias pueden procesarse en un LLM gestionado en la nube (Bedrock), en región, sin uso para entrenamiento y **con PII enmascarada** antes de la invocación.	Se reemplaza Bedrock por un modelo open-weights on-prem; el resto del diseño no cambia (el LLM está detrás de una interfaz).
S2	Los sistemas on-prem exponen APIs o SQL de lectura, y la plataforma de tickets tiene una API REST de escritura (notas, asignación, estado).	Habría que construir conectores adicionales (por ejemplo, réplica de lectura).

#	Supuesto	Impacto si no se cumple
S3	Volumen: unas 5.000 Alertas/día, que tras la correlación quedan en unas 300 Incidencias/día, con picos de 10× durante eventos masivos.	Se ajustan la concurrencia y el presupuesto.
S4	Objetivo de latencia: p95 menor a 3 min desde la Alerta hasta el resumen en el Ticket maestro.	Se ajustan los límites del agente.
S5	Existen Incidencias históricas cerradas con causa raíz y Equipo resolutor registrados (al menos unas 200 aprovechables para el golden set).	La evaluación offline empieza con un set más chico más etiquetado en modo sombra.
S6	Los procedimientos técnicos están en repositorios versionados (Git, SharePoint o Confluence) exportables a S3.	Se necesita un proceso de curación previo.
S7	El operador tiene un IdP corporativo (SAML/OIDC) y un proceso de gestión de cambios independiente.	Las acciones sugeridas no tendrían un circuito de autorización al cual derivarse.

3. Matriz de decisiones de automatización

Criterio de decisión. Si una actividad tiene una regla conocida y verificable, se resuelve de forma determinista. Se usa IA generativa solo donde la entrada es lenguaje no estructurado o la ruta de razonamiento no es predecible. El **agente** se reserva para el único caso en que el *siguiente paso depende del resultado del anterior*, que es la investigación.

#	Actividad	Automatización elegida	Por qué	Alternativa descartada
1	Recibir Alertas y Tickets	**Integración determinista**: webhook → cola → inicio del workflow	Es transporte de datos; requiere durabilidad y reintentos, no razonamiento.	LLM para "leer" eventos: costo y latencia sin aportar nada.
2	Validar campos, deduplicar y correlacionar	**Reglas** (campos obligatorios; correlación por equipo, ventana de tiempo y topología padre-hijo)	La deduplicación debe ser reproducible y auditable. La topología resuelve el 80% de la correlación (una caída upstream explica las Alertas downstream).	LLM para decidir si dos Alertas son "la misma": no determinista y difícil de auditar. Se pueden usar embeddings como señal secundaria en la fase 2, nunca como decisión.
3	Consultar cliente, servicio, ubicación, equipo y estado	**Consultas deterministas** (API/SQL)	Son lookups por clave; la respuesta correcta es única.	Que el agente lo consulte: lento, caro y no garantiza completitud.
4	Clasificar y priorizar, SLA	**Motor de reglas** (matriz impacto × urgencia, tabla de SLA) + **workflow IA** solo para extraer y normalizar campos de Tickets en texto libre	El SLA es contractual y debe ser exacto y explicable. La IA solo convierte "no tengo internet en la sede de San Isidro" en `servicio`, `ubicación` y `síntoma`, con salida estructurada.	LLM decide la prioridad: inaceptable para SLA y auditoría.
5	Investigar causas probables	**Agente IA** (loop acotado con RAG sobre procedimientos e histórico + tools de solo lectura)	La ruta de investigación depende de lo que se encuentra: si el equipo está bien se mira upstream, si hay un patrón histórico se busca su procedimiento. No es un flujo fijo.	Workflow IA de un solo paso (RAG fijo): no puede seguir pistas. Multi-agente: más costo y fallo sin más capacidad.
6	Identificar el equipo y derivar	**Tabla de ruteo** (tipo de equipo/dominio → Equipo resolutor). Si es ambiguo, la IA sugiere y el Operador NOC confirma.	El ruteo erróneo es el principal costo operativo (reasignaciones); la regla es explicable.	LLM decide el equipo: sin garantía. Se acepta solo como *sugerencia* con aprobación humana.
7	Resumen técnico	**Workflow IA** (una llamada con salida	Redactar y sintetizar es la fortaleza	Plantilla fija: no sintetiza la

#	Actividad	Automatización elegida	Por qué	Alternativa descartada
		estructurada: hechos, hipótesis, evidencias con cita, acciones sugeridas)	del LLM, y la estructura permite validarlo de forma determinista.	investigación. Agente: innecesario, no hay decisiones que tomar.
8	Actualizar el ticket y escalar	**Determinista** (escritura vía API por el workflow; reglas de escalamiento E1–E6) + **humano en el loop**	Escribir en sistemas del operador y decidir el escalamiento son acciones con impacto: deben ser predecibles.	Que el agente escriba directamente: viola el requisito de no actuar sin controles.

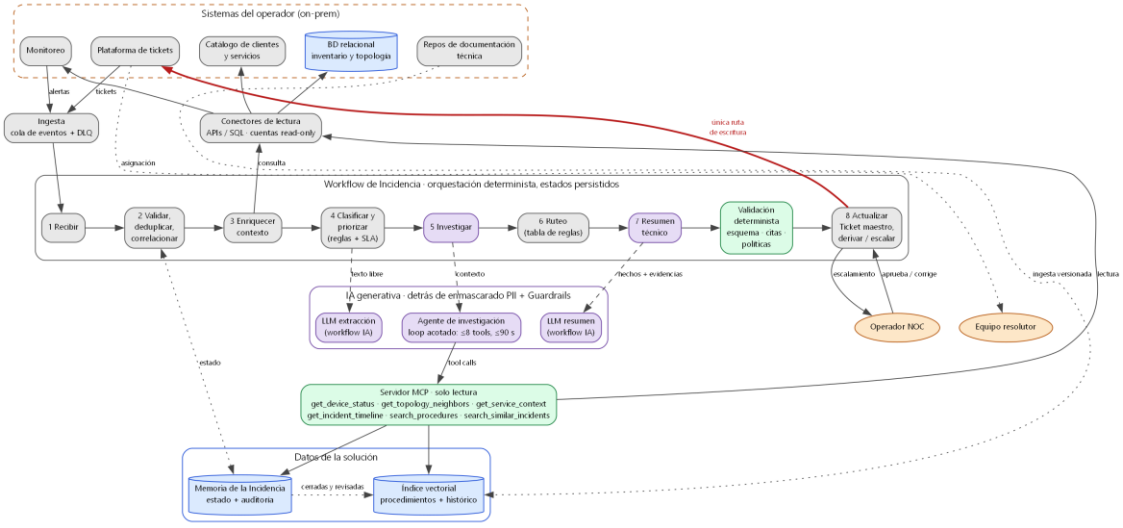
Resultado: 1 agente, 2 llamadas LLM puntuales y el resto determinista.

Señales de escalamiento al Operador NOC

Ninguna depende de una "confianza" autodeclarada por el LLM, porque no es una señal fiable. Todas son verificables:

Código	Condición	Origen
E1	Faltan campos obligatorios después de la validación y la extracción	Reglas
E2	Salida no verificable: fuera de esquema, cita inexistente o hipótesis sin evidencia	Validador determinista
E3	Ruteo ambiguo o tipo de equipo sin regla	Tabla de ruteo
E4	Prioridad P1: revisión humana obligatoria	Reglas de prioridad
E5	El agente agotó su presupuesto de pasos o tiempo (investigación incompleta)	Límites del agente
E6	Hay una acción sugerida que es un cambio y requiere autorización	Validador determinista
E7	IA no disponible: degradación (marca, no escala por sí sola)	Workflow

4. Arquitectura conceptual / lógica



Arquitectura lógica

Código de colores: gris = determinista; violeta = IA generativa; verde = control de seguridad o validación; azul = datos; naranja = personas.

4.1 Componentes, datos y límites de acceso

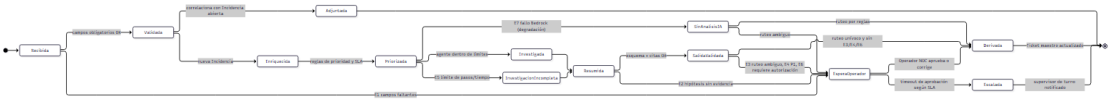
Componente	Función	Datos que recibe / entrega	Límite de acceso
Ingesta (cola + DLQ)	Desacoplar fuentes y asegurar entrega	Alertas y Tickets crudos	Solo escritura en la cola; los mensajes envenenados van a la DLQ.
Workflow de Incidencia	Orquestar las 8 actividades, persistir estados, aplicar reglas, escalar	Estado de la Incidencia	Lee vía conectores; **único componente con escritura** al Ticket maestro (vía `ticket-writer`).
Conectores de lectura	Acceder a monitoreo, catálogo, BD relacional y tickets	Datos operacionales	Cuentas de servicio **read-only** en origen.
Enmascarado PII + Guardrails	Seudonimizar clientes, IPs y contactos antes del LLM; filtrar entrada y salida	Texto → texto con tokens (`CLIENTE_3`)	El mapa de reidentificación vive en Aurora y nunca se envía al modelo.
LLM extracción (workflow IA)	Texto libre del Ticket → campos estructurados	JSON con esquema fijo	Sin tools.
Agente de investigación	Formular y contrastar hipótesis	Contexto de la Incidencia → hallazgos con `evidence_id`	Solo tools MCP de lectura; ≤8 llamadas, ≤90 s, presupuesto de tokens.
Servidor MCP	Exponer 6 tools con contrato (ver anexo)	Consultas → resultados con `evidence_id`	Rol IAM solo de lectura; sin tools de escritura.
LLM resumen (workflow IA)	Redactar el resumen técnico estructurado	Hechos y evidencias → JSON `ResumenTecnico`	Sin tools.
Validación determinista	Verificar esquema, que cada cita exista en la traza, hipótesis con evidencia y acciones de cambio marcadas	`ResumenTecnico` + traza → errores y códigos de escalamiento	No invoca el LLM.
Memoria de la Incidencia	Estado, hechos, decisiones y auditoría	Persistencia	El agente la *lee* (vía tool); solo el workflow escribe.
Índice vectorial (RAG)	Procedimientos versionados e Incidencias cerradas y revisadas	Fragmentos con documento y versión	Ingesta controlada; el agente no escribe.

4.2 Memoria

Tipo	Contenido	Ciclo de vida	Quién escribe
De trabajo	Mensajes y resultados de tools de una ejecución del agente	Se descarta al terminar; la traza queda en auditoría	Loop del agente
De la Incidencia	Hechos, hipótesis, decisiones, aprobaciones	Mientras dure la Incidencia y el período de retención	Solo el workflow, con salida validada
Institucional	Procedimientos e Incidencias cerradas, con causa raíz confirmada por humanos	Permanente y versionada	Proceso de ingesta, nunca el agente

No hay memoria que el agente escriba libremente: eso sería un vector de envenenamiento persistente.

4.3 Estados, fallos y escalamiento del workflow



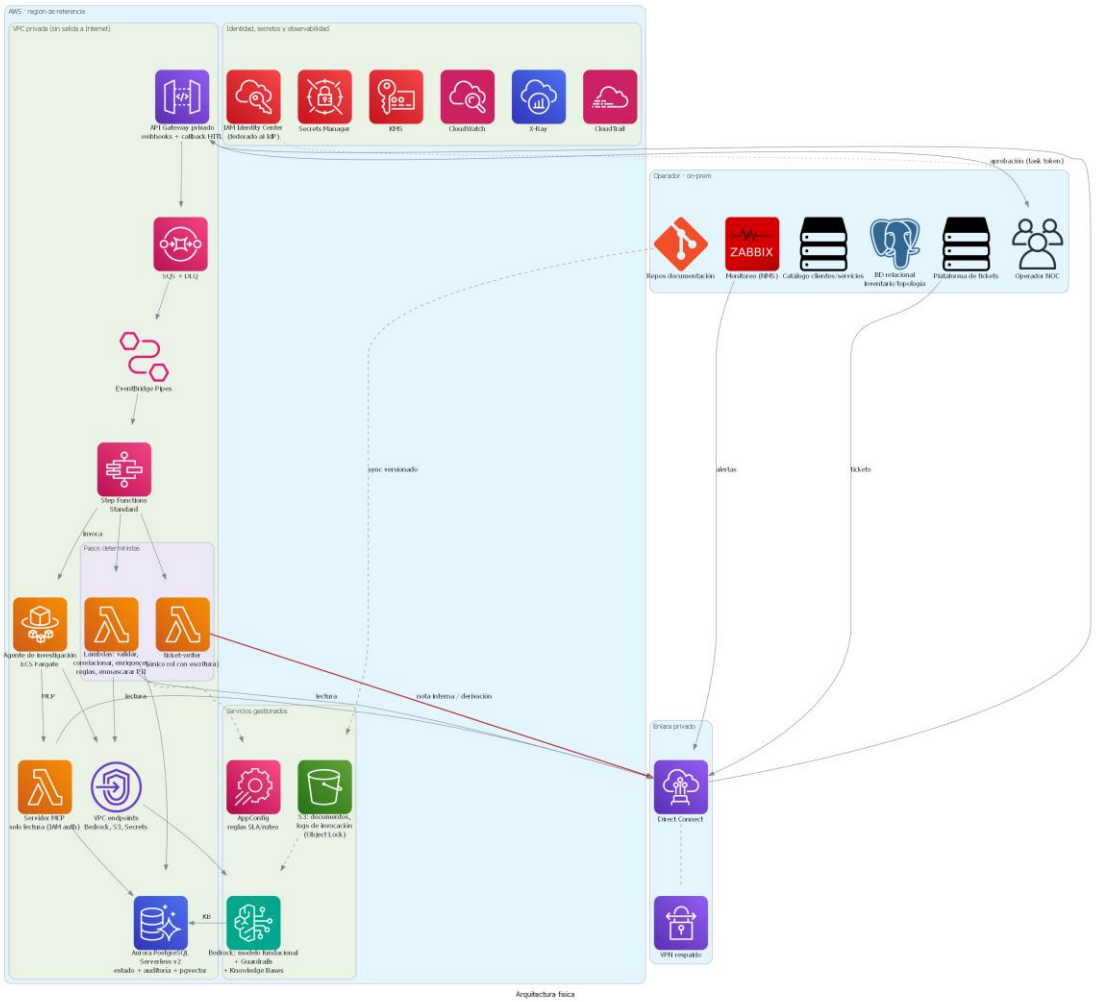
Estados del workflow

Fallo	Comportamiento
Conector on-prem caído	Reintento con backoff (3×). Si persiste, continúa con los datos disponibles, marca los "datos faltantes" en el resumen y genera E1 si el dato era obligatorio.
Bedrock con throttling o caído	Reintento con backoff. Si persiste, pasa a SinAnalisisIA (E7): derivación por reglas, o al Operador NOC si el ruteo es ambiguo.
Agente excede sus límites	Devuelve lo hallado marcado como incompleto (E5).
Salida inválida	1 reintento del resumen con los errores del validador como feedback. Si vuelve a fallar, E2.
Operador NOC no responde	Timeout ligado al SLA de la prioridad. Al vencer, escala al supervisor de turno.
Mensaje malformado	DLQ, más una alarma.

4.4 El agente de investigación

- **Entrada**: la Incidencia enriquecida y enmascarada (equipo, servicios, Alertas correlacionadas y prioridad).
- **Tools**: `get_device_status`, `get_topology_neighbors`, `get_service_context`, `get_incident_timeline`, `search_procedures` y `search_similar_incidents`. Todas son de lectura y devuelven un `evidence_id`.
- **Salida**: hallazgos y una sugerencia de equipo, cada afirmación con su `evidence_id`.
- **Límites**: máximo 8 tool calls, 90 s y un presupuesto de tokens. El contenido proveniente de Tickets y documentos va delimitado como *dato no confiable* en el prompt.
- **No puede**: escribir en ningún sistema, ejecutar comandos ni decidir la prioridad o el escalamiento.

5. Arquitectura física (AWS, híbrida)



Arquitectura física

Necesidad	Servicio	Justificación	Alternativa descartada
Conectividad con el operador	**Direct Connect** + Site-to-Site VPN de respaldo	Latencia estable y tráfico privado hacia sistemas críticos.	Solo VPN: aceptable para un piloto, no para producción.
Ingesta	**API Gateway privado → SQS** (+DLQ) → **EventBridge Pipes**	Absorbe picos 10× (S3), con reintentos y DLQ, sin código de pegamento.	Kinesis: sobredimensionado para unas 5.000 Alertas/día.
Orquestación	**Step Functions Standard**	Estados explícitos, historial auditable, reintentos y catch por estado, `waitForTaskToken` para HITL (ADR 0001).	Temporal o LangGraph: un servicio más que operar.
Pasos deterministas	**Lambda** en VPC	Tareas cortas y sin estado; escalan a cero.	Contenedores: innecesario para pasos de segundos.
Reglas de SLA y ruteo	**AppConfig**	Las reglas cambian por negocio: despliegue validado, versionado y con rollback sin redeploy.	Reglas hardcodeadas: cada cambio requiere un release.

Necesidad	Servicio	Justificación	Alternativa descartada
Agente	**ECS Fargate** + Bedrock Converse API (loop propio)	Ejecuciones de hasta ~90 s con límites explícitos y trazables (ADR 0002).	Bedrock Agents: loop opaco. Lambda: viable, pero Fargate da margen para picos de latencia.
Tools (MCP)	**Lambda** detrás de **API Gateway privado con autenticación IAM**	Frontera de seguridad separada con un rol de solo lectura (ADR 0003).	Tools dentro del agente: el agente tendría las credenciales.
Modelos	**Amazon Bedrock**: un modelo fundacional de nivel pequeño (extracción) y uno de nivel medio (agente y resumen), con versión fijada. Opciones en Bedrock: Anthropic Claude, Amazon Nova, Meta Llama o Mistral; la elección final se hace con el golden set de §7 (calidad en español, tool use, costo y latencia).	Modelos gestionados en región, sin uso de datos para entrenamiento, sin GPU que operar. Dos niveles para optimizar costo. La Converse API es común a todos los modelos, así que cambiar de modelo no cambia el código.	Self-hosting en GPU: mayor costo y operación; queda como plan B ante S1.
Guardrails	**Bedrock Guardrails**	Detección de PII, temas denegados y prompt attacks como segunda capa tras el enmascarado propio.	Solo enmascarado propio: sin defensa en profundidad.
RAG	**Bedrock Knowledge Bases** sobre **Aurora PostgreSQL + pgvector**, embeddings multilingües (Cohere Embed Multilingual en Bedrock)	Ingesta y chunking gestionados; pgvector reutiliza el mismo Aurora. Embeddings multilingües porque la documentación está en español.	OpenSearch Serverless: costo mínimo alto para este volumen.
Estado y auditoría	**Aurora PostgreSQL Serverless v2**	Memoria de la Incidencia, auditoría y vectores en un solo motor relacional.	DynamoDB: consultas de auditoría y joins más difíciles.
Documentos y logs	**S3** con SSE-KMS; logs de invocación del modelo con **Object Lock**	Fuentes del RAG y auditoría inmutable de prompts y respuestas.	—
Identidad	**IAM Identity Center** federado al IdP del operador; un rol IAM por componente	SSO corporativo para humanos; mínimo privilegio por servicio.	Usuarios IAM locales.
Secretos y cifrado	**Secrets Manager** (rotación) + **KMS** (claves propias)	Credenciales de conectores fuera del código; cifrado en reposo.	—
Red	VPC privada sin salida a Internet; **VPC endpoints** para Bedrock, S3 y Secrets Manager	El tráfico al modelo no sale a Internet.	NAT Gateway: abre una ruta de exfiltración.
Observabilidad	**CloudWatch** (métricas, logs, alarmas), **X-Ray** (trazas), **CloudTrail** (API), Bedrock invocation logging	Nativos y suficientes; trazas de negocio en Step Functions.	Langfuse o similar: mejora futura para análisis de calidad de prompts.

Región. La región de referencia es us-east-1, por disponibilidad de modelos en Bedrock. Si el operador exige residencia en Latinoamérica, hay que validar antes de producción la disponibilidad de los modelos y de Knowledge Bases en sa-east-1 o mx-central-1 (ver §8).

6. Seguridad: matriz de riesgos y controles

Marco: **OWASP Top 10 for LLM Applications (2025)** más riesgos clásicos de plataforma. Cada control indica **cómo se verifica**.

Riesgo	Escenario	Controles	Verificación
LLM01 Prompt injection (indirecta)	El texto de un Ticket o de un documento instruye "reinicia el router".	El agente no tiene tools de escritura (ADR 0003); el contenido externo va delimitado como dato; Guardrails contra prompt attacks; las acciones de cambio requieren autorización.	Caso C07 del prototipo; IAM Access Analyzer sobre el rol MCP (sin acciones de escritura); red-team periódico.
LLM06 Agencia excesiva	El agente ejecuta acciones sobre la red o el ticket.	Rol del agente: solo `execute-api` sobre MCP y `bedrock:InvokeModel`. La escritura está aislada en `ticket-writer`.	Policy-as-code en CI (cfn-guard o Checkov): el build falla si el rol del agente tiene `Put*`, `Update*` o `Delete*`.
LLM02 Fuga de información sensible	Datos de clientes en prompts, logs o respuestas.	Seudonimización previa; Guardrails PII; región fija; VPC endpoints; logs cifrados con KMS.	Test que verifica que ningún prompt registrado contiene patrones PII (escaneo de muestras con Macie sobre el bucket de logs).
LLM09 Desinformación / alucinación	Una hipótesis inventada se presenta como hecho.	Esquema que separa hechos de hipótesis; cada afirmación lleva un `evidence_id` verificado contra la traza; si falla, E2.	Casos C05, C06 y C13; métrica de groundedness en la evaluación offline.
LLM05 Manejo inseguro de la salida	La salida del LLM se usa sin validar (ruteo, campos, HTML en el ticket).	Validación Pydantic; la salida nunca decide prioridad ni SLA; escape de contenido al escribir en el ticket.	Tests deterministas en CI.
LLM04 / LLM08 Envenenamiento de datos o embeddings	Un documento o Incidencia histórica manipulada sesga al agente.	Solo se ingestan Incidencias cerradas *y revisadas por humanos*; fuentes con control de versiones y aprobación; metadato de versión en cada fragmento.	Auditoría de la ingesta (quién y qué versión); la evaluación de recuperación detecta derivas.
LLM10 Consumo desmedido	Loop del agente, picos o abuso que disparan el costo.	Límites de pasos, tiempo y tokens; concurrencia máxima por cola; cuotas de Bedrock.	Alarmas de CloudWatch de costo por Incidencia y tokens por hora; AWS Budgets.
LLM03 Cadena de suministro / cambio de modelo	Un cambio de versión del modelo degrada la calidad sin aviso.	Versión de modelo fijada; un cambio de modelo o prompt pasa por el gate de evaluación.	Pipeline: no se despliega si la regresión baja de los umbrales de §7.
LLM07 Fuga del system prompt	Exposición de la lógica interna.	El prompt no contiene secretos ni reglas sensibles; las reglas viven en AppConfig.	Revisión de prompts en code review.
Acceso no autorizado	Un usuario aprueba o modifica Incidencias sin permiso.	SSO federado; aprobación ligada a la identidad del ticket; callback HITL con task token de un solo uso.	Registro de auditoría en Aurora + CloudTrail; test de callback con token inválido.
Credenciales expuestas	Las credenciales on-prem se filtran.	Secrets Manager con rotación; cuentas read-only en origen.	Rotación automática verificada; escaneo de secretos en el repo.
Disponibilidad	Caída del LLM o de un conector.	Degradación (E7); DLQ; Direct Connect + VPN.	Pruebas de caos: simular el corte de Bedrock y verificar la derivación por reglas (caso C12).

7. Estrategia de evaluación de la IA generativa

7.1 Qué se evalúa

Componente	Qué se mide
Extracción (paso 4)	Exactitud por campo frente a etiquetas humanas.
Recuperación RAG	Si trae el procedimiento y la Incidencia similar correctos.
Agente (paso 5)	Si llega a la causa raíz correcta, con qué eficiencia y respetando los límites.
Resumen (paso 7)	Fidelidad a la evidencia, completitud y utilidad para el Operador NOC.
Sistema completo	Tiempo hasta triage, calidad del ruteo y costo.

7.2 Tres capas

Capa 1: determinista (en CI, en cada cambio de prompt, modelo, reglas o código).

- Validez de esquema, existencia de citas en la traza, hipótesis con evidencia y acciones de cambio marcadas.
- Reglas de prioridad, SLA, ruteo y escalamiento con casos tabulados.
- Casos adversariales: prompt injection, campos faltantes, Alertas duplicadas e IA caída.
- **Implementado en el prototipo**: 15 casos, todos pasan (`eval_deterministic.py`). Las salidas del modelo en esos casos son fijas: el prototipo verifica el *validador y las reglas*, no la calidad de un modelo real.

Capa 2: offline probabilística (golden set de unas 200 Incidencias históricas anonimizadas más casos sintéticos).

Métrica	Componente	Método	Umbral inicial propuesto
Exactitud por campo	Extracción	Comparación exacta con la etiqueta	≥ 95%
Recall@3 / MRR	Recuperación	Documento o Incidencia relevante en el top 3	Recall@3 ≥ 0,85
Causa raíz top-1 / top-3	Agente	Comparación con la causa confirmada (LLM-as-judge + revisión humana en muestra)	top-3 ≥ 0,70
Pasos promedio y % de E5	Agente	Traza	E5 < 10%
Groundedness / fidelidad	Resumen	LLM-as-judge con rúbrica, **calibrado contra etiquetas humanas** (concordancia $\kappa \geq 0,7$ antes de confiar en él)	≥ 0,90
Accuracy de ruteo	Ruteo + sugerencia IA	Comparación con el Equipo resolutor final	≥ 0,90
Tasa de bloqueo de inyección	Seguridad	Casos adversariales	100% sin efectos

Cada evaluación se corre varias veces ($n \geq 3$) para estimar la varianza. Un cambio se acepta si no hay regresión significativa.

Capa 3: online (en producción).

Métrica	Señal
Tiempo hasta triage (objetivo del reto)	Desde la Alerta hasta la derivación, comparado con la línea base actual.
Tasa de aceptación del resumen	Aprobaciones sin edición del Operador NOC.
Distancia de edición del resumen	Cuánto corrige el operador.
Tasa de reasignación	Derivaciones erróneas.
Tasa de escalamiento por código (E1–E7)	Detecta derivas: por ejemplo, un aumento de E2 indica que el modelo cita peor.
Costo y latencia por Incidencia	p50/p95 y USD por Incidencia.

Las correcciones del Operador NOC alimentan el golden set (con revisión), lo que cierra el ciclo de mejora.

7.3 Despliegue gradual

1. **Sombra** (4–6 semanas): la IA genera el análisis pero no se publica. El Operador NOC trabaja como hoy y se comparan resultados. Pasa a la fase 2 si se cumplen los umbrales de la capa 2 sobre datos reales.
2. **Asistido**: el resumen se publica como nota interna; la derivación automática ocurre solo con ruteo unívoco; el resto requiere aprobación.
3. **Nunca autónomo** para acciones sobre la red: siempre pasan por la gestión de cambios del operador.

8. Operación y limitaciones

8.1 Latencia y costo (estimación)

Paso	Latencia estimada	Tokens estimados por Incidencia
Pasos 1–4 deterministas	< 5 s	—
Extracción (modelo pequeño, solo Tickets)	1–2 s	~1,5k entrada / 0,3k salida
Agente (modelo medio, ≤8 tools)	30–90 s	~30k entrada / 2k salida (acumulado)
Resumen (modelo medio)	5–10 s	~8k entrada / 1k salida
Total hasta el resumen	**~1–2 min (p95 < 3 min)**	

Con precios de lista de referencia de USD 3 / 15 por millón de tokens (entrada/salida) para el modelo medio y USD 1 / 5 para el pequeño, el costo de LLM ronda **USD 0,15 por Incidencia**, unos **USD 45/día** a 300 Incidencias/día. *Los precios deben verificarse contra la lista vigente de Bedrock.* Prompt caching sobre el system prompt y los contratos de tools lo reduciría. El costo de infraestructura (Aurora Serverless, Fargate, Direct Connect) es aparte y está dominado por Direct Connect.

8.2 Observabilidad

- **Por Incidencia**: historial de Step Functions + tabla de auditoría (quién, qué, con qué evidencia, versión de prompt y modelo).

- **Por invocación LLM**: Bedrock invocation logging (S3 con Object Lock) + métricas de tokens y latencia.
- **Alarmas**: DLQ no vacía, tasa E7 > 0, costo por hora, p95 > 3 min, aumento de E2.

8.3 Resiliencia

La IA es un acelerador, no una dependencia crítica: con Bedrock caído la operación continúa por reglas (E7). La ingesta es durable (SQS + DLQ) y Step Functions reintenta por estado. El enlace con el operador es redundante (Direct Connect + VPN).

8.4 Supervisión humana

- El Operador NOC aprueba todo lo que cae en E1–E6, todas las P1 y toda sugerencia de ruteo hecha por IA.
- Las acciones de cambio nunca se ejecutan: se derivan a la gestión de cambios.
- Las correcciones del operador quedan registradas y alimentan la evaluación.

8.5 Riesgos residuales

- **Correlación imperfecta**: Incidencias sin relación topológica conocida pueden quedar separadas. Se mitiga con revisión del operador y, más adelante, con embeddings como señal secundaria.
- **El validador de citas comprueba que la evidencia existe, no que la respalde.** Una cita real puede usarse para sostener una conclusión incorrecta. Se mitiga con groundedness en la evaluación y con revisión humana.
- **Detección de acciones de cambio por palabras clave** (respaldo del validador): puede tener falsos negativos con redacciones atípicas. Debe evaluarse en el golden set.
- **Calidad del histórico**: si las causas raíz registradas son pobres, el RAG de Incidencias similares aporta poco.
- **Dependencia del enmascarado**: un identificador no reconocido podría llegar al modelo. Guardrails es la segunda capa, no una garantía.

8.6 Pendientes antes de producción

1. Confirmar S1 con el área legal y de cumplimiento del operador (residencia de datos y uso de un LLM gestionado).
2. Validar la disponibilidad de los modelos y de Knowledge Bases en la región exigida.
3. Inventariar las APIs reales de los sistemas on-prem y la plataforma de tickets (S2).
4. Construir y etiquetar el golden set con el equipo NOC; acordar los umbrales.
5. Definir con negocio la matriz de prioridad, el SLA y la tabla de ruteo reales (las del prototipo son supuestos).
6. Prueba de carga con picos 10× y prueba de caos de Bedrock y de los conectores.
7. Pentest y red-team de prompt injection antes de la fase asistida.

9. Decisiones técnicas principales

ADR	Decisión
0001	Workflow determinista con un único agente de investigación (no un agente orquestador ni multi-agente).
0002	Loop de agente propio sobre la Bedrock Converse API, con límites duros (no Bedrock Agents).
0003	Servidor MCP de solo lectura como frontera de seguridad; la escritura solo la hace el workflow.

Otras decisiones: reglas fuera del LLM (prioridad, SLA, ruteo y escalamiento); escalamiento por señales verificables en lugar de "confianza" del modelo; degradación sin IA; despliegue sombra → asistido.

Anexo A. Prototipo

En la carpeta `prototipo/`:

Archivo	Contenido
`tools.json`	Contratos (JSON Schema) de las 6 tools MCP, todas de solo lectura y con `evidence_id` en la salida.
`resumen.py`	Esquema Pydantic `ResumenTecnico` y el validador determinista (citas, hipótesis, acciones de cambio).
`reglas.py`	Campos obligatorios, prioridad/SLA, ruteo y códigos de escalamiento (valores supuestos).
`casos.json`	15 casos: normales, de escalamiento, adversariales (prompt injection, cita alucinada, acción disfrazada) y degradación.
`eval_deterministic.py`	Corre los casos; sale con código 1 si alguno falla (gate de CI).

Ejecución: `uv run --python 3.13 --with pydantic python eval_deterministic.py` → `15/15 casos OK`.

No implementado: el loop del agente, el servidor MCP, la definición ASL de Step Functions, la infraestructura y cualquier llamada a un modelo real.